

20 Essential Coding Interview Problems

Clean, Well-Commented Python Solutions • Ready for Technical Interviews

What's Inside — Carefully written Python solutions for 20 of the most frequently asked coding interview problems. Each solution includes clear function signatures with type hints, inline comments explaining the logic, efficient algorithms suitable for interviews, and alternative approaches where relevant. Perfect for last-minute revision and interview preparation.

Table of Contents

01. Reverse a String (without slicing)
02. Check if a String is a Palindrome
03. Find Factorial of a Number (Recursive + Iterative)
04. Generate Fibonacci Series
05. Check if a Number is Prime
06. Find Duplicates in a List
07. Count Frequency of Elements / Characters
08. Two Sum Problem
09. First Non-Repeating Character in a String
10. Merge Two Sorted Lists
11. Flatten a Nested List
12. Swap Two Numbers without a Temporary Variable
13. Check if Two Strings are Anagrams
14. FizzBuzz Problem
15. Find Maximum / Minimum without Built-in Functions
16. Remove Duplicates while Preserving Order
17. Sort a Dictionary by Values
18. Implement Binary Search
19. Detect Cycle in a Linked List
20. Find the Middle of a Linked List

Problem 01 — Reverse a String (without slicing)

Reverse a string without using Python's slicing syntax (`[::-1]`). Two approaches are shown: iterative concatenation and the two-pointer technique.

```
def reverse_string(s: str) -> str:
    """Reverse a string without using slicing."""
    reversed_str = ""
    for char in s:
        reversed_str = char + reversed_str # Prepend each character
    return reversed_str

# Alternative using two pointers
def reverse_string_two_pointers(s: str) -> str:
    chars = list(s)
    left, right = 0, len(chars) - 1
    while left < right:
        chars[left], chars[right] = chars[right], chars[left]
        left += 1
        right -= 1
    return "".join(chars)
```

Problem 02 — Check if a String is a Palindrome

Determine whether a string reads the same forwards and backwards. Optionally ignores case and spaces.

```
def is_palindrome(s: str) -> bool:
    """Check if a string reads the same forwards and backwards."""
    s = s.lower().replace(" ", "") # Optional: ignore case & spaces
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

Problem 03 — Find Factorial of a Number (Recursive + Iterative)

Compute the factorial of a non-negative integer using both recursive and iterative approaches.

```
def factorial_recursive(n: int) -> int:
    """Calculate factorial using recursion."""
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n: int) -> int:
    """Calculate factorial using iteration."""
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

Problem 04 — Generate Fibonacci Series

Generate the first n numbers of the Fibonacci sequence where each number is the sum of the two preceding ones.

```
def fibonacci(n: int) -> list:
    """Generate first n Fibonacci numbers."""
    if n <= 0:
        return []
    if n == 1:
        return [0]

    series = [0, 1]
    for i in range(2, n):
        next_num = series[i - 1] + series[i - 2]
        series.append(next_num)
    return series
```

Problem 05 — Check if a Number is Prime

Efficiently determine whether a given integer is a prime number using optimized trial division up to the square root.

```
def is_prime(n: int) -> bool:
    """Check whether a number is prime."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False

    # Check divisibility up to sqrt(n)
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

Problem 06 — Find Duplicates in a List

Identify all elements that appear more than once in a list, returning them as a unique list of duplicates.

```
def find_duplicates(lst: list) -> list:
    """Return a list of duplicate elements."""
    seen = set()
    duplicates = set()
    for item in lst:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)
```

Problem 07 — Count Frequency of Elements / Characters

Count how many times each element (or character) appears. Works for both lists and strings.

```
def count_frequency(items) -> dict:
    """Count frequency of each element (works for lists or strings)."""
    freq = {}
    for item in items:
        freq[item] = freq.get(item, 0) + 1
    return freq

# Using collections.Counter (more pythonic)
from collections import Counter

def count_frequency_counter(items) -> dict:
    return dict(Counter(items))
```

Problem 08 — Two Sum Problem

Given an array of integers and a target, find the indices of the two numbers that add up to the target. Uses a hash map for O(n) time.

```
def two_sum(nums: list, target: int) -> list:
    """
    Return indices of the two numbers that add up to target.
    Assumes exactly one solution exists.
    """
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return [] # No solution found
```

Problem 09 — First Non-Repeating Character in a String

Find the first character in a string that appears exactly once. Returns None if every character repeats.

```
def first_non_repeating(s: str) -> str:
    """Return the first character that appears only once."""
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1

    for char in s:
        if freq[char] == 1:
            return char
    return None # No non-repeating character found
```

Problem 10 — Merge Two Sorted Lists

Merge two already-sorted lists into a single sorted list in linear time without using built-in sort.

```
def merge_sorted_lists(list1: list, list2: list) -> list:
    """Merge two sorted lists into one sorted list."""
    merged = []
    i = j = 0

    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            merged.append(list1[i])
            i += 1
        else:
            merged.append(list2[j])
            j += 1

    # Append remaining elements
    merged.extend(list1[i:])
    merged.extend(list2[j:])
    return merged
```

Problem 11 — Flatten a Nested List

Recursively flatten a nested list of arbitrary depth into a single flat list.

```
def flatten(nested_list: list) -> list:
    """Recursively flatten a nested list of any depth."""
    result = []
    for item in nested_list:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
```

Problem 12 — Swap Two Numbers without a Temporary Variable

Swap the values of two variables using arithmetic or bitwise XOR, without using a temporary variable.

```
def swap_without_temp(a: int, b: int) -> tuple:
    """Swap two numbers using arithmetic (no temp variable)."""
    a = a + b
    b = a - b
    a = a - b
    return a, b

# Using XOR (works for integers)
def swap_xor(a: int, b: int) -> tuple:
    a = a ^ b
    b = a ^ b
    a = a ^ b
    return a, b
```

Problem 13 — Check if Two Strings are Anagrams

Determine whether two strings contain the same characters with the same frequencies (anagrams of each other).

```
def are_anagrams(s1: str, s2: str) -> bool:
    """Check if two strings are anagrams of each other."""
    s1 = s1.lower().replace(" ", "")
    s2 = s2.lower().replace(" ", "")

    if len(s1) != len(s2):
        return False

    return sorted(s1) == sorted(s2)

# Using frequency count (more efficient for large strings)
def are_anagrams_freq(s1: str, s2: str) -> bool:
    s1 = s1.lower().replace(" ", "")
    s2 = s2.lower().replace(" ", "")

    if len(s1) != len(s2):
        return False

    freq = {}
    for char in s1:
        freq[char] = freq.get(char, 0) + 1
    for char in s2:
        if char not in freq or freq[char] == 0:
            return False
        freq[char] -= 1
    return True
```

Problem 14 — FizzBuzz Problem

Classic FizzBuzz: for numbers 1 to n, print 'Fizz' for multiples of 3, 'Buzz' for multiples of 5, and 'FizzBuzz' for multiples of both.

```
def fizzbuzz(n: int) -> list:
    """
    Return a list from 1 to n where:
    - Multiples of 3 → "Fizz"
    - Multiples of 5 → "Buzz"
    - Multiples of both → "FizzBuzz"
    """
    result = []
    for i in range(1, n + 1):
        if i % 15 == 0:
            result.append("FizzBuzz")
        elif i % 3 == 0:
            result.append("Fizz")
        elif i % 5 == 0:
            result.append("Buzz")
        else:
            result.append(str(i))
    return result
```

Problem 15 — Find Maximum / Minimum without Built-in Functions

Find the maximum and minimum values in a list without using Python's built-in max() or min() functions.

```
def find_max_min(lst: list) -> tuple:
    """Return (maximum, minimum) without using max() or min()."""
    if not lst:
        raise ValueError("List is empty")

    maximum = minimum = lst[0]
    for num in lst[1:]:
        if num > maximum:
            maximum = num
        if num < minimum:
            minimum = num
    return maximum, minimum
```

Problem 16 — Remove Duplicates while Preserving Order

Remove duplicate elements from a list while keeping the original order of first occurrences.

```
def remove_duplicates(lst: list) -> list:
    """Remove duplicates while preserving the original order."""
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

Problem 17 — Sort a Dictionary by Values

Return a new dictionary whose items are sorted by their values (ascending or descending).

```
def sort_dict_by_values(d: dict, reverse: bool = False) -> dict:
    """Return a new dictionary sorted by its values."""
    # sorted() returns a list of (key, value) tuples
    sorted_items = sorted(d.items(), key=lambda item: item[1], reverse=reverse)
    return dict(sorted_items)
```

Problem 18 — Implement Binary Search

Classic binary search on a sorted array. Returns the index of the target or -1 if not found. Time complexity: $O(\log n)$.

```
def binary_search(arr: list, target: int) -> int:
    """
    Perform binary search on a sorted list.
    Returns the index of target, or -1 if not found.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Problem 19 — Detect Cycle in a Linked List

Detect whether a singly linked list contains a cycle using Floyd's Tortoise and Hare algorithm (two pointers).

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def has_cycle(head: ListNode) -> bool:
    """
    Detect if a linked list has a cycle using
    Floyd's Tortoise and Hare algorithm.
    """
    if not head or not head.next:
        return False

    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next           # Move one step
        fast = fast.next.next      # Move two steps
        if slow == fast:
            return True           # Cycle detected
    return False
```


Problem 20 — Find the Middle of a Linked List

Find the middle node of a singly linked list in one pass using the two-pointer technique. For even length, returns the second middle node.

```
def find_middle(head: ListNode) -> ListNode:
    """
    Find the middle node of a linked list.
    If even number of nodes, returns the second middle node.
    """
    if not head:
        return None

    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next          # Move one step
        fast = fast.next.next     # Move two steps

    return slow # slow is now at the middle
```

End of Document • Practice these problems regularly for interview success.